COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	08/18/26	**DATE SUBMITTED:**	08/20/26
NAME:	Medina, Jomz Ryle E.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 6.1 Searching Techniques

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Create C++ code for searching data using different techniques

We will implement our searching technique on a data filled with random values. To begin, we want the following directives:

```
#include <iostream>
#include <cstdlib> //for generating random integers
#include <time.h> //will be used for our seeding function
```

Then, we will define the capacity of data elements in our dataset.

```
const int max_size = 50;
```

This will be used to define your dataset. Your driver code must have the following:

```
//generate random values
int dataset[max_size];
srand(time(0));
for(int i = 0; i < max_size; i++){
    dataset[i] = rand();
}
//show your datasets content
for(int i = 0; i < max_size; i++){
    std::cout << dataset[i] << " ";
}
```

Once you are sure that you can generate random integers for your dataset, you can delete the second iterative loop and move on with the creation of the different searching techniques. **Verify this by including a screenshot of the generated data upon successful implementation of the code above in table 6-1.**

This implementation will also require the use of linked lists, so create a file "nodes.h" in your C++ project which will contain the definition of our single linked list node:

```
template <typename T>
class Node{
public:
    T data;
    Node *next;
};
```

Followed by a function that will provide us with the address of an object that points to null.

```
template <typename T>
Node<T> *new_node(T newData){
    Node<T> *newNode = new Node<T>;
    newNode->data = newData;
    newNode->next = NULL;
    return newNode;
}
```

Once this has been made, you are free to proceed to the next parts.

A.1 Sequential Search

Create a separate file implementation called "searching.h" which we will use for the implementation of the algorithms below.

```
LINEAR SEARCH ALGORITHM
N -> Boundary of the list
Item -> Searching number
Data -> Linear array
Step 1: I := 0
Step 2: Repeat while I <= n
  If (item = data[i])
  Print "Searching is successful"
  Exit
Else
  Print "Searching is Unsuccessful"
Exit
```

Create a function called linearSearch that accepts necessary parameters to execute the algorithm described above. Indicate your code, output, and observations in table 6-2a.

Create a linked list that has your first name per node's data.

```
//create linked list for linear search
Node<char> *name1 = new_node('R');
Node<char> *name2 = new_node('o');
Node<char> *name3 = new_node('m');
Node<char> *name4 = new_node('a');
Node<char> *name5 = new_node('n');
```

Then, link each node to each other.

```
//linked list
name1->next = name2;
name2->next = name3;
name3->next = name4;
name4->next = name5;
name5->next = NULL;
```

Create a sequential search algorithm for linked list by modifying the earlier function called linearLS. This function must take the head node of your linked list and a variable dataFind that it must search for within the given list.

```
//linear search
linearLS(name1, 'n');
```

The sequential search algorithm does not change even if it operates on a linked list. **Show your modified code from earlier so that it will operate on a linked list. Screenshot your code and output with corresponding observations/notes in table6-2b.**

B.1 Binary Search

This algorithm must also be included in your "searching.h" file.

```
BINARY SEARCH ALGORITHM
Step 1: low :=0, up=n-1
Step 2: Repeat while low <= up
    mid = int(low+up)/2
  if(no=arr[mid])
    print "Search element is found!"
    exit
    else
  if(no < arr[mid]) then
     up =  mid-1
else
     low = mid+1
Step 3: Print "Search element is not found"
Step 4: Exit
```

Create a function called binarySearch that accepts necessary parameters to execute the algorithm described above. Indicate your code, output, and observations in table 6-3a.

Next, just like in the linked list with linear search, we will perform binary search in a linked list. However, there are pre-requisites for this to work: the linked list must have ordered values and we must know how many nodes exist in your list. For our testing, we can create our linked list with the following code:

```
//create a linked list for binary search
//make sure the values are ordered
char choice = 'y'; int count = 1; int newData;
Node<int> *temp, *head, *node;

while(choice=='y'){
    std::cout << "Enter data: ";
    std::cin >> newData;

    if(count==1){
        head = new_node(newData);
        std::cout << "Successfully added " << head->data << " to the list.\n";
        count++;
    } else if(count==2) {
        node = new_node(newData);
        head->next = node;
        node->next = NULL;
        std::cout << "Successfully added " << node->data << " to the list.\n";
```

```
            count++;
        } else {
            temp = head;
            while(true){
                if(temp->next == NULL) break;
                temp = temp->next;
            }
            node = new_node(newData);
            temp->next = node;
            std::cout << "Successfully added " << node->data << " to the list.\n";
            count++;
        }

        //when to end
        std::cout << "Continue? (y/n)";
        std::cin >> choice;
        if(choice=='n') break;
    }
```

An optional snippet of code can be added here to verify that our data is actually stored onto a linked list as intended.

```
//display
Node<int> *currNode;
currNode = head;
while(currNode!=NULL){
    std::cout << currNode->data << " ";
    currNode = currNode->next;
}
```

Binary search still works the same, however. We need to create a function that will enable us to find the middle node. This function, getMiddle, must take the first and last node as its parameters. It follows the algorithm described below:

1. Traverse the singly linked list using two pointers.
2. Move one pointer by one step ahead and the other pointer by two steps.
3. When the fast pointer reaches the end of the singly linked list, the slow pointer will reach the middle of the singly linked list.
4. Return slow pointer address.

We will use this function to find the middle node of our linked list to properly search through the sorted list. Next, another function to implement the binary search. Step 2 of this algorithm uses the function from the algorithm above.

1. Start node is set to head of the list and the last node is set to NULL.
2. Middle element is calculated using the two pointers approach discussed above.
3. If the middle element is same as the key to be searched, we return it.
4. Else if middle element is greater than the key to be searched, we have to search is the right side of the singly linked list. So, we set start pointer to the next of middle element.
5. Else if middle element is less than the key to be searched, we have to search is the left side of the singly linked list. So, we set last pointer to the middle element.
6. If the key is found or the entire linked list gets traversed, we break the loop.

Create the C++ code for this binary search algorithm and provide the output in the three cases shown in table 6-3b. Screenshot and observation are required.

OUTPUT:

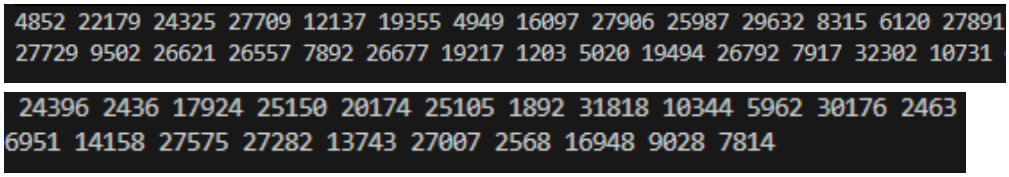
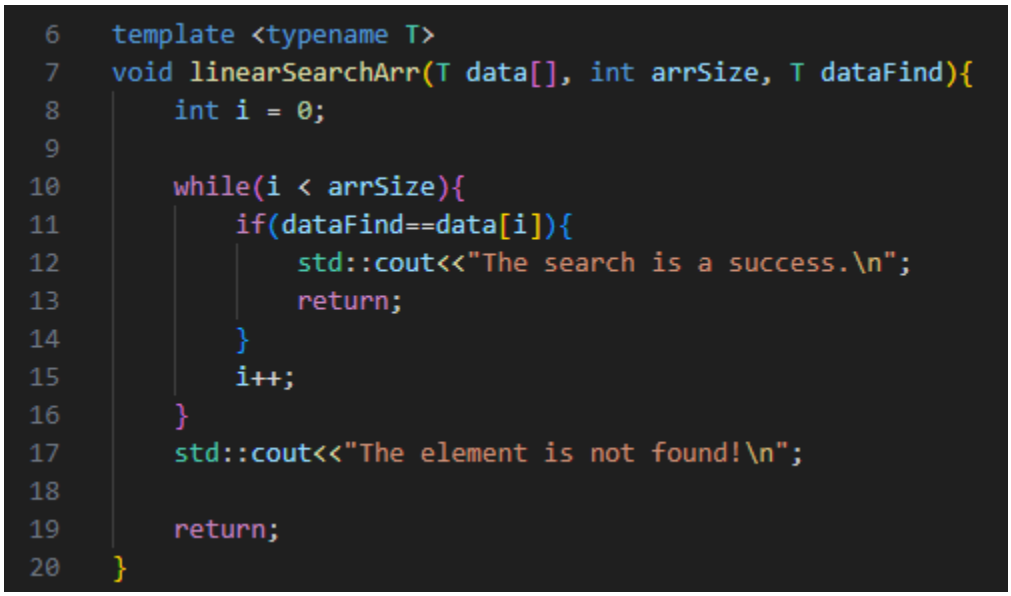
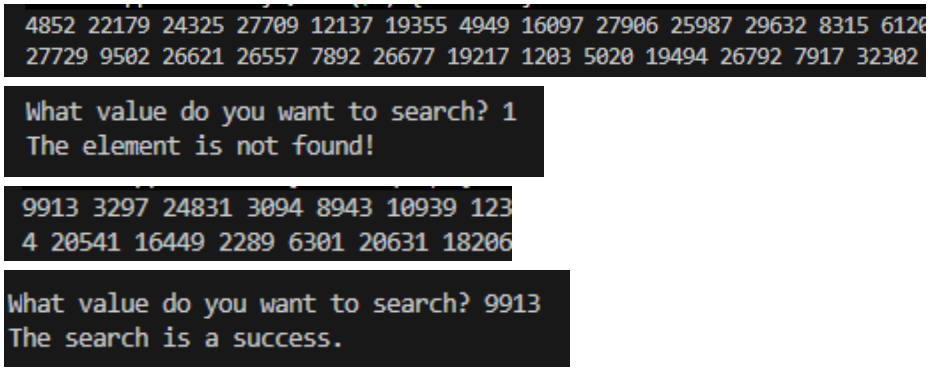
Screenshot	
Observations	When the code shown in the lab (as well as through the online lecture) is executed, the following terminal output is shown. It simply creates an array and fills it with randomly generated numbers using the function, **rand()** that is seeded with the use of current time. In the second loop, it simply prints out the numbers in the said created array but formatted properly by separating them with spaces.

Table 6-1. Data Generated and Observations

Code	
Output	
Observations	In this code, to put it in the most simplest terms in my own understanding, the function shown in our **searchingAlgo.h** library does a linear search by checking every item in the array individually from the start towards the finish. It first finds a match for the target value that we provide, followed by printing a message for success if it does and saying no if it doesn't.

```
template <typename T>
void linearSearchArr(T data[], int arrSize, T dataFind){
    int i = 0;

    while(i < arrSize){
        if(dataFind==data[i]){
            std::cout<<"The search is a success.\n";
            return;
        }
        i++;
    }
    std::cout<<"The element is not found!\n";

    return;
}
```

```
4852 22179 24325 27709 12137 19355 4949 16097 27906 25987 29632 8315 6126
27729 9502 26621 26557 7892 26677 19217 1203 5020 19494 26792 7917 32302

What value do you want to search? 1
The element is not found!

9913 3297 24831 3094 8943 10939 123
4 20541 16449 2289 6301 20631 18206

What value do you want to search? 9913
The search is a success.
```

Table 6-2a. Linear Search for Arrays

Code

(searchingAlgo.h)

```
22  template <typename T>
23  void linearSearchLL(Node<T>*node, T dataFind){
24      while(node!=nullptr){
25          if(dataFind == node->data){
26              std::cout<<"The search was a success.\n";
27              return;
28          }
29          node = node->next;
30      }
31      std::cout<<"The search was unsuccessful!\n";
32      return;
33  }
```

(main.cpp)

```
29      // Linked List:
30      Node<char>* name1 = new_node('J');
31      Node<char>* name2 = new_node('O');
32      Node<char>* name3 = new_node('M');
33      Node<char>* name4 = new_node('Z');
34
35      name1->next = name2;
36      name2->next = name3;
37      name3->next = name4;
38      name4->next = NULL;
39
40      std::cout<<std::endl;
41
42      std::cout<<"Linear Search for Linked List Result:\n";
43      linearSearchLL(name1, 'J');
44      linearSearchLL(name1, 'A');
```

Output

```
Linear Search for Linked List Result:
The search was a success.
The search was unsuccessful!
```

Observations

The following code creates a linked list that stores my first name, **"J, O, M, Z"** before passing it through our created **linearSearchLL()** function that, just like with the previous code, searches through a linked list instead of an array this time around. It does it by comparing the data to find with the data stored within the node and looping through by changing where the node is pointed to. Of course, if it isn't found, a message appears confirming that.

Table 6-2b. Linear Search for Linked List

Code

searchingAlgo.h:

```
52  template <typename T>
53  void binarySearchArr(T arr[], int n, T no) {
54      int low = 0;
55      int up = n - 1;
56
57      while (low <= up) {
58          int mid = (low + up) / 2;
59
60          if (no == arr[mid]) {
61              std::cout << "The search is a success.\n";
62              return;
63          } else {
64              if (no < arr[mid]) {
65                  up = mid - 1;
66              } else {
67                  low = mid + 1;
68              }
69          }
70      }
71
72      std::cout << "The search was unsuccessful!\n";
73  }
```

main.cpp:

```
18      // BUBBLE SORT (sorting so that binary search works)
19      for (int i = 0; i < max_size - 1; i++) {
20          for (int j = 0; j < max_size - i - 1; j++) {
21              if (dataset[j] > dataset[j + 1]) {
22                  // Swap elements
23                  int temp = dataset[j];
24                  dataset[j] = dataset[j + 1];
25                  dataset[j + 1] = temp;
26              }
27          }
28      }
```

(Bubble sort added in between dataset generation and printing out so array-based binary search works)

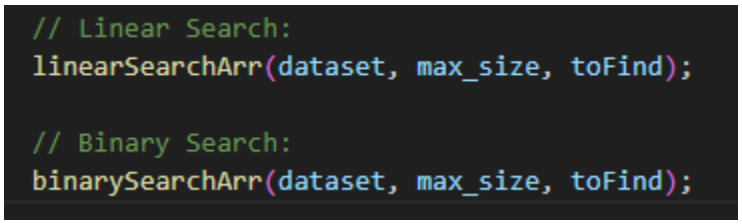
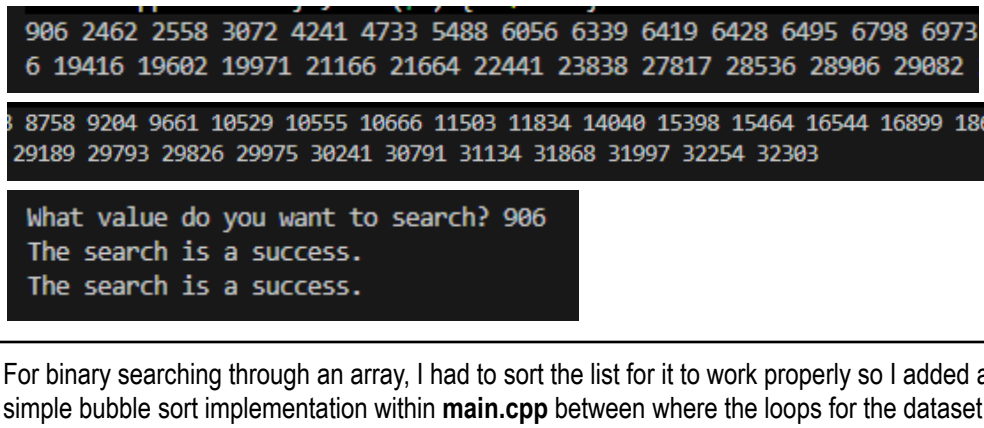
	
Output	
Observations	For binary searching through an array, I had to sort the list for it to work properly so I added a simple bubble sort implementation within **main.cpp** between where the loops for the dataset is generated and when it is printed. The code I created for the binary searching function follows the dividing and conquering concept that is highlighted in the lab where instead of checking every single item, we check for the middle number and cut what we have to search left in each go-through. Here, in the output, it is shown that the search is a success with **'906'** as it is the first element shown in the dataset.

Table 6-3a. Binary Search for Arrays

Code

```
52  template <typename T>
53  void binarySearchLL(Node<T> *start, T findData){
54      Node<T>* last = nullptr;
55
56      while(start!=last){
57          Node<T>* middle = getMiddle(start, last);
58          if(middle == nullptr){
59              return;
60          }
61
62          if(middle->data == findData){
63              std::cout<<"The search was a success.\n";
64              return;
65          }
66
67          if(middle->data < findData){
68              start = middle->next;
69          } else {
70              last = middle;
71          }
72      }
73      std::cout<<"The search was unsuccessful!\n";
74  }
75
```

```
48      // Implement it here:
49      std::cout<<"Binary Search for Linked List Result:\n";
50      binarySearchLL(name1, 'M');
51      binarySearchLL(name1, 'A');
52      return 0;
53  }
```

Output

```
Binary Search for Linked List Result:
The search was success.
The search was unsuccessful!
```

Observations

Finally, the code above shows our binary search but adapted for a linked list based implementation. Following the lab, I made use of a helper function, getMiddle(), where we repeatedly find the node at the center in order to make the searching range smaller. It changes the pointers for **start** as well as **end** that depends on whether the target we have is bigger or smaller than the value of the current middle node. In the outputs, searching for **'M'** in **'J, O, M, Z'** succeeds as it is present, and searching for **'A'** fails as it is clearly not there.

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve different problems utilizing appropriate searching techniques in C++

For each provided problem, give a screenshot of your code, the output console, and your answers to the questions.

Problem 1. Suppose you are doing a sequential search of the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]. Utilizing both a linked list and an array approach to the list, use sequential search and identify how many comparisons would be necessary to find the key '18'?

Problem 2. Modify your sequential search algorithm so that it returns the count of repeating instances for a given search element 'k'. Test on the same list given in problem 1.

Problem 3. Suppose you have the following sorted list [3, 5, 6, 8, 11, 12, 14, 15, 17, 18] and are using the binary search algorithm. If you wanted to find the key 8, draw a diagram that shows how the searching works per iteration of the algorithm. Prove that your drawing is correct by implementing the algorithm and showing a screenshot of the code and the output console.

Problem 4. Modify the binary search algorithm so that the algorithm becomes recursive. Using this new recursive binary search, implement a solution to the same problem for problem 3.

Question 1	
Code	(searchingAlgo.h - modified):

```
6   template <typename T>
7   void linearSearchArr(T data[], int arrSize, T dataFind){
8       int i = 0;
9
10      while(i < arrSize){
11          if(dataFind==data[i]){
12              std::cout<<"The search is a success.\nNumber of comparisons: "<<i+1<<"\n";
13              return;
14          }
15          i++;
16      }
17      std::cout<<"The element is not found!\nNumber of comparisons: "<<arrSize<<"\n";
18
19      return;
20  }
21
22  template <typename T>
23  void linearSearchLL(Node<T>*node, T dataFind){
24      int i = 0; // number of comparisons
25      while(node!=nullptr){
26          i++;
27          if(dataFind == node->data){
28              std::cout<<"The search was a success.\nNumber of comparisons: "<<i<<"\n";
29              return;
30          }
31          node = node->next;
32      }
33      std::cout<<"The search was unsuccessful!\nNumber of comparisons: "<<i<<"\n";
34      return;
35  }
```

supplementary.cpp:

main.cpp | supplementary.cpp | main.exe

supplementary.cpp > main()

```cpp
#include <iostream>
#include "node.h"
#include "searchingAlgo.h"

int list[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
int max_size = sizeof(list)/sizeof(list[0]);
int toFind = 18;

int main() {

    // Linked-List Creation
    Node<int>* name1 = new_node(15);
    Node<int>* name2 = new_node(18);
    Node<int>* name3 = new_node(2);
    Node<int>* name4 = new_node(19);
    Node<int>* name5 = new_node(18);
    Node<int>* name6 = new_node(0);
    Node<int>* name7 = new_node(8);
    Node<int>* name8 = new_node(14);
    Node<int>* name9 = new_node(19);
    Node<int>* name10 = new_node(14);


    name1->next = name2;
    name2->next = name3;
    name3->next = name4;
    name4->next = name5;
    name5->next = name6;
    name6->next = name7;
    name7->next = name8;
    name8->next = name9;
    name9->next = name10;
    name10->next = NULL;

    // SEARCHES:

    // Array Based Sequential/Linear Search:
    std::cout<<"Array-Based Linear Search: ";
    linearSearchArr(list, max_size, toFind);

    // Linked-List Based Sequential/Linear Search:
    std::cout<<"Linked List-Based Linear Search: ";
    linearSearchLL(name1, 18);

    return 0;
}
```

Output

```
Array-Based Linear Search: The search is a success.
Number of comparisons: 2
Linked List-Based Linear Search: The search was a success.
Number of comparisons: 2
```

Answer	The comparisons needed to find '**18**' in the given dataset with the use of both searches is **2**. For question 1, I decided to just modify the functions for linear searches on arrays as well as linked lists that we've already created in the main procedure. It was fairly easy as I simply made use of the variable '**i**' that was already being put into work for the index in accessing the given array (adding one so that it correctly matches the amount of comparisons actually done) and merely made my own '**i**' with the linked-list based implementation (also incrementing it on my own and shoving it alongside the success message).

Question 2

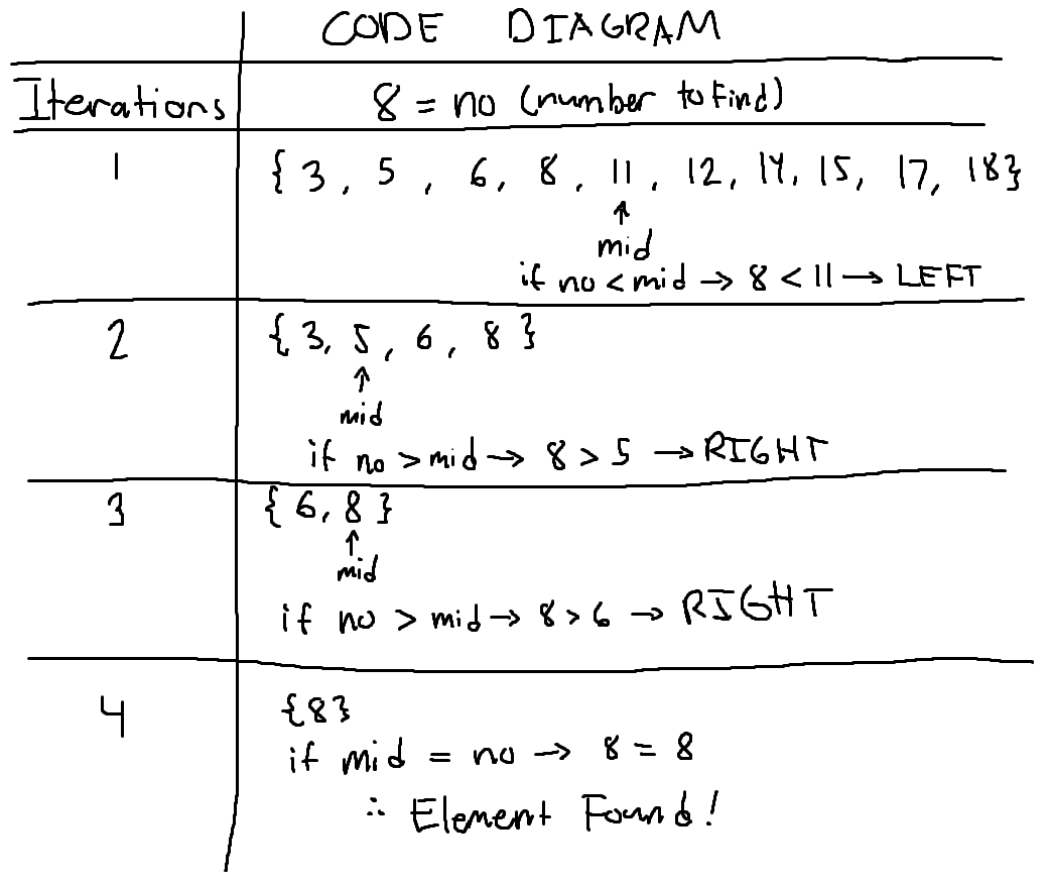
Code

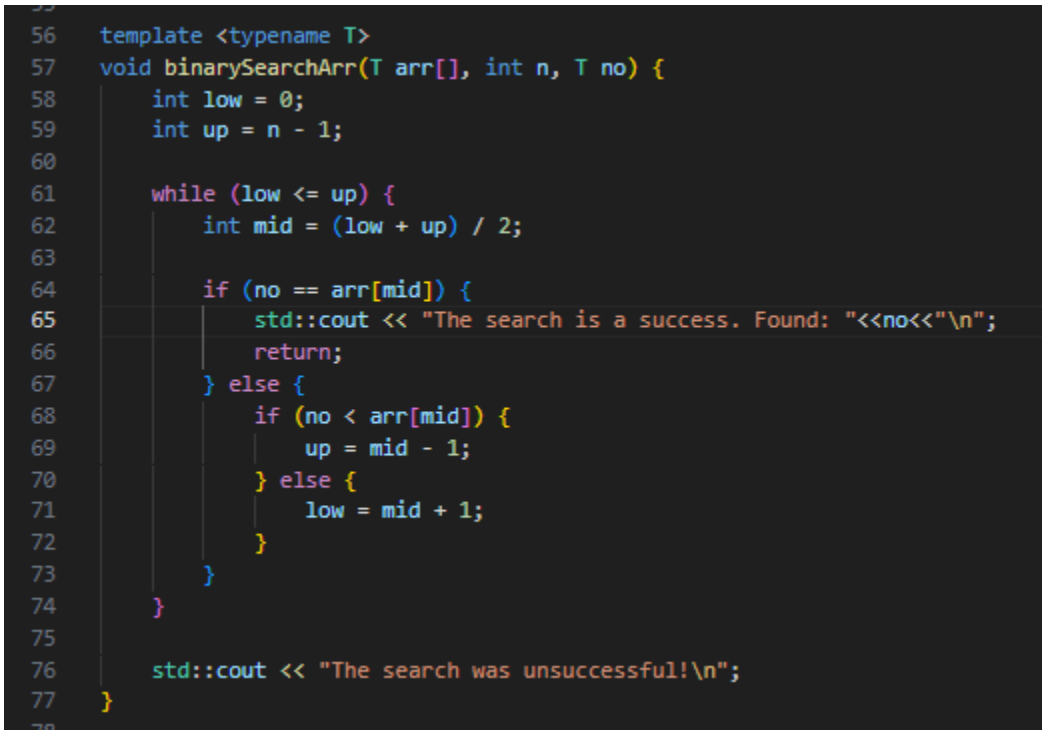
```
6   template <typename T>
7   void linearSearchArr(T data[], int arrSize, T dataFind){
8       int j = 0; // comps
9       int i = 0; // counts
10
11      while(i < arrSize){
12          if(dataFind==data[i]){
13              std::cout<<"Element found! Number of comparisons: "<<i+1<<"\n";
14              j++;
15          }
16          i++;
17      }
18      std::cout<<"The element '"<<dataFind<< "' is found "<<j<<" time/s.\n";
19
20      return;
21  }
22
23  template <typename T>
24  void linearSearchLL(Node<T>*node, T dataFind){
25      int i = 0; // number of comparisons
26      int j = 0; // count
27      while(node!=nullptr){
28          i++;
29          if(dataFind == node->data){
30              std::cout<<"Element found! Number of comparisons: "<<i<<"\n";
31              j++;
32          }
33          node = node->next;
34      }
35      std::cout<<"The element '"<<dataFind<< "' is found "<<j<<" time/s.\n";
36      return;
37  }
```

Output	Array-Based Linear Search: Element found! Number of comparisons: 2 Element found! Number of comparisons: 5 The element '18' is found 2 time/s. Linked List-Based Linear Search: Element found! Number of comparisons: 2 Element found! Number of comparisons: 5 The element '18' is found 2 time/s.
Answer	The modified code checks through the entire array/linked list instead of returning early, keeping track of comparisons and the specific count with added variables through the loops. This gives us the moment when each element is found at a specific comparison as well as the total times that the element is found throughout the list. Giving us '**18**' being found **2** times.

Question 3

Diagram



Code	
Output	The search is a success. Found: 8
Answer	The searching works by the sorted list being cut in half repeatedly and checking if the number we want is smaller or larger than the value in the middle. In the first iteration, we start at 11 which goes down to 3-8, going down to 6-8, and finally reaching the specific point of 8 at the fourth iteration.

```
56  template <typename T>
57  void binarySearchArr(T arr[], int n, T no) {
58      int low = 0;
59      int up = n - 1;
60
61      while (low <= up) {
62          int mid = (low + up) / 2;
63
64          if (no == arr[mid]) {
65              std::cout << "The search is a success. Found: "<<no<<"\n";
66              return;
67          } else {
68              if (no < arr[mid]) {
69                  up = mid - 1;
70              } else {
71                  low = mid + 1;
72              }
73          }
74      }
75
76      std::cout << "The search was unsuccessful!\n";
77  }
```

Question 4	

Code	(searchingAlgo.h)
	`template <typename T>`
	`void binarySearchArr(T arr[], int low, int high, T no) {`
	`  if (low > high) {`
	`  std::cout<<"The search was unsuccessful!\n";`
	`  }`
	`  int mid = (low + high) / 2;`
	`  if (arr[mid] == no){`
	`  std::cout<<"The search was a success.\n";`
	`  return;`
	`  } else if (no < arr[mid]){`
	`  binarySearchArr(arr, low, mid-1, no);`
	`  } else {`
	`  binarySearchArr(arr, mid+1, high, no);`
	`  }`
	`}`
	(supplementary.cpp)
	`std::cout<<"Recursive Binary Search: ";`
	`binarySearchArr(list2, 0, max_size-1, toFind2);`

Output	`Recursive Binary Search: The search was a success.`
Answer	The new modified **binarySearchArr()** function makes use of converting the original binary search into a recursive function that repeatedly calls itself with high as well as the low bounds updated through each call that cuts the search range in half until the middle element eventually matches the target value. Additionally, it follows the same logic in dividing and conquering where instead of iterative loops, we're making use of repeated function calls.

Conclusion:
In this activity, I learned a lot about how linear as well as binary searching algorithms handle different data structures such as with arrays and linked lists. That linear searching looks at elements one by one which has its strengths in unsorted lists or through counting values that repeat and how binary searching is also strong when it comes to sorted data sets which are efficient in finding the element we need through the principle of dividing and conquering. Additionally, I've also seen how to properly implement these searches with loops as well as recursive functions. Overall, I believe I have a firmer grasp on the respective efficiencies of the searching algorithms and their most base-concepts. I believe I can have more improvements in my total understanding of them, but I now have a foundation on what to recall when I need these algorithms when dealing with finding specific elements in datasets.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **None**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.